Affiliation: FAST NUCES, Main campus.			
Department: Artificial Intelligence		**Specialization:** Agentic AI **Domain** Artificial Intelligence	
Proposal Title: LedgerGuard: A Multi-Agent System for Invoice and Contract Reconciliation			
Required Budget **US dollars** 0	**Proposed Duration** **(Max months)** About 1.25 months (5 weeks, tentative)	**Desired Starting Date** 5 November 2026	
Team members	**Academic Title**	**Investigator's Affiliation (Dept. / College)**	**Domain**
Syed Muneeb Ur Rehman (23K-0038)	Undergrad student	BSAI - FAST NUCES	Artificial Intelligence
Adina Faraz (23K-0008)	Undergrad student	BSAI - FAST NUCES	Artificial Intelligence
External Investigator None			

Abstract

Before an invoice is paid, accounts-payable staff must check it against the purchase order, goods receipt and vendor contract. Much of this work is still manual, and mistakes can cause overpayment, duplicate payment or payment to a fraudulent account. LedgerGuard is a proposed multi-agent system that will extract data from these documents, find the relevant contract terms, run rule-based checks in Python and cite the source of each finding. A verification step will reject any finding that its source does not support. Each invoice will then go to simulated clearance or to a human reviewer, depending on its risk. Two undergraduate students will build the prototype in about five weeks, using text-based PDFs and simulated actions. The prototype will be evaluated on a synthetic benchmark, with discrepancy detection F1 score, false-approval rate, evidence verification accuracy, attack success rate, review workload and latency as the measures. The system has not been implemented yet, so no results are reported.

Keywords:

accounts payable; three-way matching; multi-agent systems; agentic RAG; evidence grounding; human-in-the-loop; prompt injection

1. Introduction

Before a company pays a supplier, accounts-payable (AP) staff confirm that the invoice agrees with the purchase order and goods receipt, a check known as the three-way match. They also compare it with the prices in the vendor contract. An invoice may overstate a quantity or price, use an expired contract rate, miss an agreed discount, state the wrong payment terms or be submitted twice. Tater et al. describe this process as manual, labour-intensive and error-prone, and estimate about ten minutes of manual work per invoice [1].

Fraud is a second concern, since an attacker can send a genuine-looking invoice with changed bank details. Large language models (LLMs) can read varied document layouts, but they can also make claims that their sources do not support, and they can be misled by instructions hidden in a document. For this reason, a system that informs payment decisions should not rely on an LLM's answer alone.

LedgerGuard, a semester project by two undergraduate students, addresses this problem. Since it has not been built yet, all outcomes described here are expected outcomes.

2. Literature Review

Tater et al. describe a deployed AP system that combines business rules, fuzzy matching and information retrieval to check duplicates, vendors and line items, including matching against purchase orders and goods receipts [1]. It processed about 80,000 invoices, and 76% of them needed little or no manual intervention. However, the paper gives no precision or recall for individual modules, and its savings are estimates. The system does not use LLM agents, and the modules it describes do not cover contract terms.

Thomas et al. apply multi-agent LLMs to finance, using a planner, a supervisor and specialist agents to investigate financial anomalies [2]. Their framework reached 66% per-run accuracy, compared with 52% for a single agent. However, its latency was about five times higher, and it was tested on only 20 synthetic scenarios. The framework also starts after an anomaly has been flagged, so it does not detect discrepancies itself. These results suggest that agents can help but add cost, so each agent needs a clear purpose.

These two studies leave a gap that this project can explore. One checks invoices without LLM agents or contract terms, and the other uses agents only after detection. Neither reports verifying each finding against a cited text span or testing prompt injection inside documents. Because this review covers only two sources, we do not claim that LedgerGuard is novel. Our aim is a small, repeatable study of this combination on synthetic data.

3. Description (Rationale) of the Proposed Work

The idea came from a project request we saw on Upwork, a freelance marketplace. No client is involved, and the posting is not used as evidence of wider demand.

An agent-based design suits this problem because it involves different kinds of tasks. Reading documents and finding clauses need judgement, but adding up line items should give the same answer every time. LLMs will therefore handle reading, retrieval, explanation and evidence checking, while ordinary code will handle calculations, routing and approval rules.

We reduced the original concept so that it fits two students and about five weeks. The supervisor will be rule-based, vendor memory will be a SQLite table, and one local MCP server with simulated actions will replace live Gmail and Google Sheets. A simple review page will replace React.

The project will address four questions. How accurately can the pipeline detect seeded discrepancies, and how often does it wrongly clear a problematic invoice? Does the verification step reduce unsupported findings and false approvals? Does tool-based contract retrieval find

the correct clause more often than fixed top-k retrieval? How well do design-level controls limit prompt injection inside documents?

4. Research Objectives

Tables 4.1 and 4.2 map the objectives, approaches, phases, and the tasks carried out to implement the objectives.

The eight objectives are as follows.

Objective 1: Build a multi-agent workflow in which a rule-based supervisor routes each document set through extraction, contract analysis, reconciliation and fraud-signal steps.

Objective 2: Develop a contract agent with hierarchical retrieval tools and compare it with fixed top-k retrieval.

Objective 3: Attach a page and text-span citation to each field and finding, and use an observer agent to reject unsupported claims.

Objective 4: Implement checks for quantity, unit price, rate validity, volume discount, payment terms and duplicates as tested Python functions.

Objective 5: Create a simple time-aware vendor memory of bank-account history and past dispute outcomes.

Objective 6: Implement auto, review and escalate tiers with hard override rules and a pause-and-resume approval step.

Objective 7: Expose simulated actions through a local MCP server, so that no payment or message tool runs without an approval record.

Objective 8: Evaluate the system on about 100 synthetic document sets and a set of adversarial cases.

Table 4.1 OBJECTIVES AND THEIR APPROACH MAPPING

Objective	Approach for achieving the objective
1	Build a LangGraph state graph with fixed routing rules and a log of each step.
2	Build section, search and clause-reading tools over a Pinecone index, then run both retrieval methods on the same labelled questions.
3	Parse PDFs with PyMuPDF and validate outputs with Pydantic. Check that each cited span exists, then ask a separate LLM prompt whether it supports the claim.
4	Write each check as a Python function with unit tests. The LLM may explain results but will not calculate them.
5	Store vendor facts in SQLite with valid-from and recorded-at dates, and compare invoice bank details against them.
6	Compute a deterministic score from checks and signals. Tune the concept's score bands (0-19, 20-49, 50+) on development data only. Use LangGraph interrupts for approval.
7	Use the MCP Python SDK for read-only lookups and for simulated actions that verify an approval record before running.
8	Generate labelled data, freeze a test split, run three configurations and the attack cases, and report results with confidence intervals.

Table 4.2 OBJECTIVE PHASES AND TASKS MAPPING

Objectives	Phases	Tasks
1, 4	Phases 4 and 6	Define workflow state and routing. Connect all steps. Write and unit-test the check functions. Add step logging and failure handling.
2	Phases 1 and 4	Review retrieval methods. Build contract retrieval tools. Label clause questions. Prepare the fixed top-k baseline.
3	Phases 3 and 4	Parse PDFs. Define extraction schemas. Add citations. Build span and support checks.
5	Phases 2 and 4	Design the vendor table. Generate vendor history. Add bank-change and look-alike name signals.
6, 7	Phases 4 and 6	Implement scoring and hard rules. Add pause and resume. Build the review page and local MCP server. Test that unapproved actions are refused.
8	Phases 2, 5 and 7	Generate and label the benchmark. Create attack cases. Run configurations. Analyse results and write the report.

5. Scope

In scope: English, text-based PDF invoices, purchase orders, goods receipts and contracts, including simple amendments; one currency per document set; the checks in Objective 4; bank-detail and look-alike name signals; a simple review page; simulated actions; and evaluation on synthetic data.

Out of scope: scanned documents, real company data, live ERP, payment or email connections, production deployment, a user study, and any claim of regulatory compliance or real-world fraud detection performance.

Possible extensions, if time remains, are a React interface and tests on scanned documents, but these are not commitments.

6. Research Design

We will build a working prototype and test it in a controlled comparison on labelled synthetic data. The independent variables are the use of the verification step and the retrieval method, and the dependent variables are the metrics in Section 7. Because the data is synthetic, the results cannot establish performance on real company documents.

The project will consist of the following phases:

6.1 Phase 1: Comprehensive Literature Review

We will review relevant work, keep a reference log and confirm our design choices in Week 1.

6.2 Phase 2: Data Collection

A Python generator will produce about 100 labelled document sets, with about 30 for development and 70 for testing. Some sets will be clean, while others will contain seeded discrepancies. About 20 further cases will contain injected instructions. The final size may change depending on API limits.

6.3 Phase 3: Data Pre-processing (Feature Extraction)

In this project, feature extraction means extracting fields from documents. PyMuPDF will keep page and position data, and an LLM will convert the text into records checked against Pydantic schemas. Each value must point to its source text.

6.4 Phase 4: Deep Learning Model and Probabilistic Graphical Model

LedgerGuard will not train a deep learning or probabilistic graphical model, because it will use existing LLMs through an API. Instead, this phase covers the supervisor workflow, contract agent, deterministic checks, fraud signals, vendor memory, observer and risk scoring.

6.5 Phase 5: Evaluation

The test split will be frozen before the final runs. We will test the full system, the system without the observer, and the system with fixed retrieval. Attack cases will run against the full system.

6.6 Phase 6: Tool Development

We will build the local MCP server, the approval step and a simple Streamlit review page that shows each finding beside its cited text.

6.7 Phase 7: Writing-up and Documentation

We will document the code, benchmark and experiments, and write the final report alongside the evaluation.

7. Research Methodology

Figure 7.1 shows the proposed workflow, a simplified version of our initial concept. The rule-based supervisor sends each document set through the steps in order, because later steps depend on earlier ones.

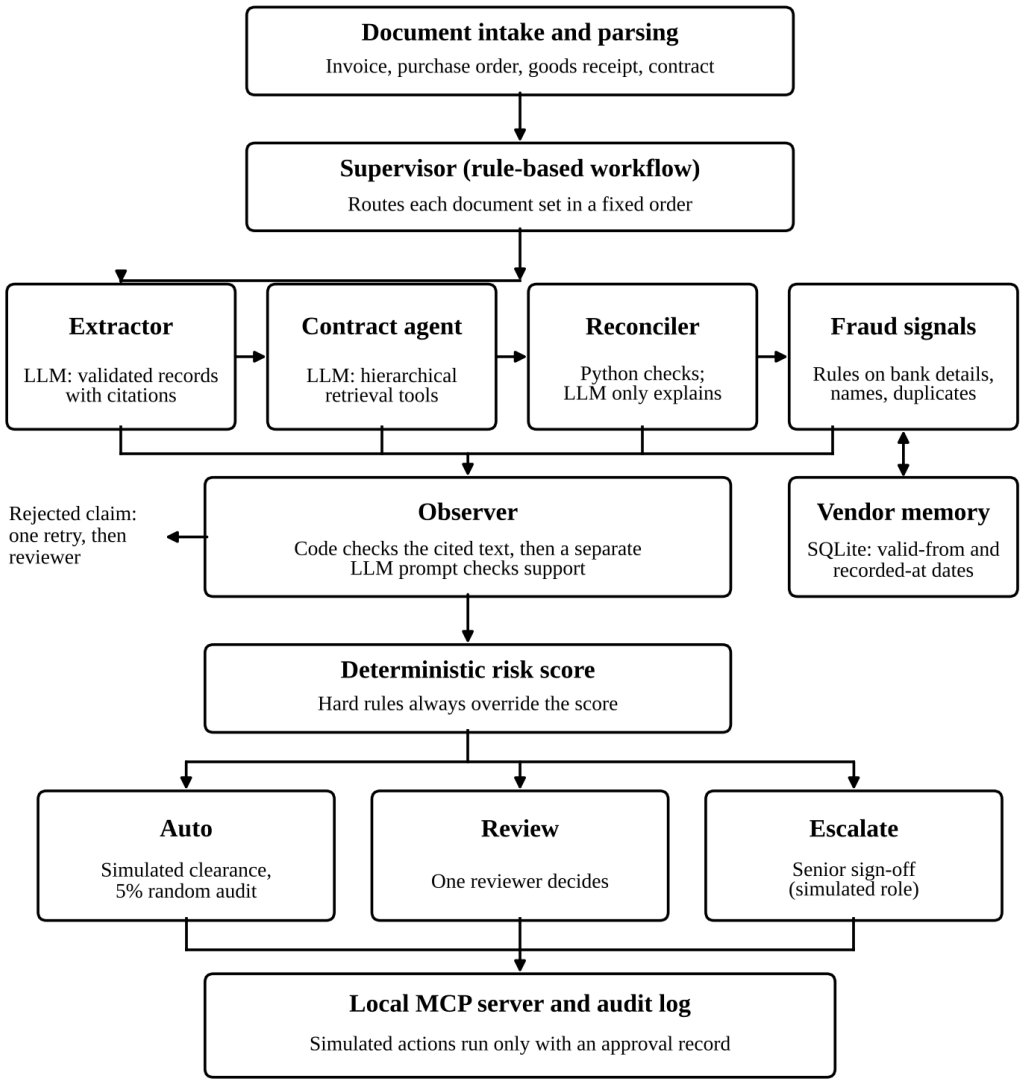


Figure 7.1 Proposed LedgerGuard workflow

Extractor. An LLM converts parsed text into validated records with a citation for each field.

Contract agent. An LLM uses tools to list sections, search and read clauses, and check amendment dates.

Reconciler. Python functions run the checks, and an LLM may only explain the results.

Fraud signals. Rules compare bank details with vendor memory, compare vendor names by string similarity and look for duplicates.

Observer. Code first confirms that the cited text exists on the cited page and that quoted numbers match. A separate LLM prompt then judges whether the text supports the claim. A rejected claim is retried once and then goes to a reviewer, marked as unverified.

Risk routing. A deterministic score sets the tier. Low-risk invoices receive simulated clearance with a 5% random audit, medium-risk invoices go to one reviewer, and high-risk invoices need a simulated senior sign-off. A changed bank account, suspected injection or unverified finding always goes to a person. Reviewers can approve or reject a case, and decisions are stored in vendor memory and the audit log.

Security. Document text is treated as untrusted data. Agents that read documents have no access to action tools, and their output must fit a fixed schema. Only ordinary code can call the MCP action tools, which refuse any request without an approval record.

Tools. The prototype will use Python, LangGraph, Pydantic, PyMuPDF, Pinecone, SQLite, the MCP Python SDK, Streamlit, pytest and the free tier of the Groq LLM API. Unstructured, LlamaIndex, FastAPI and React are dropped or deferred because the smaller scope does not need them.

Evaluation. We chose metrics that the benchmark labels can measure. We will measure extraction by field accuracy, discrepancy detection by precision, recall and F1 score, and clause retrieval by hit rate at k. False-approval rate is the share of sets with a true discrepancy that receive simulated clearance. This metric matters most, because no person would see those errors. Evidence verification accuracy will be measured on supported and unsupported claims, including corrupted citations. Attack success rate is the share of adversarial cases that change a finding or trigger a forbidden action. Review workload is the share of sets sent to a person. We will also record latency per set.

Because the test set is small, we will report counts with 95% Wilson confidence intervals. We do not promise target scores, and results on seeded signals will not be treated as real-world fraud detection performance.

8. Management & Working Plans with Team Roles

8.1 Management Plan, Research Team, Roles, and Tasks

The two members will share design decisions, meet weekly and keep their work in a shared repository. If the schedule slips, we will cut vendor memory depth first, then review-page features, then benchmark size. The role division below is a proposal for the team to confirm. Any papers written from this work would be optional additions to the project. The full papers will be included in the final report as appendices of the report.

Tables 8.1 and 8.2 introduce and detail the research team, their roles, the tasks carried out by the team members and their timelines.

Table 8.1 ROLE AND INVOLVEMENT DURATION OF RESEARCH TEAM

Team Members	**Role**	**Duration** (months)
Senior Personnel:	Dr. Muhammad Farrukh Shahid	-
Adina Faraz (23K-0008)	Leads document parsing and extraction, deterministic checks, risk scoring, the MCP server and the document generator. Shares literature review, integration, evaluation and documentation.	1.25 (tentative)
Syed Muneeb Ur Rehman (23K-0038)	Leads contract retrieval, the observer, fraud signals and vendor memory, the review page, and benchmark labels and attack cases. Shares literature review, integration, evaluation and documentation.	1.25 (tentative)

Table 8.2 PROJECT TIMELINE MAPPED WITH THE TEAM ROLES AND PHASES

Phase	**Lead**	**W1**	**W2**	**W3**	**W4**	**W5**
1. Literature Review	Both	X				
2. Data Collection	Both	X	X			
3. Data Pre-processing (Feature Extraction)	Adina	X	X			
4. Model phase: agent pipeline and checks (see 6.4)	Both		X	X		
5. Evaluation	Both				X	X
6. Tool Development	Adina (MCP); Muneeb (review page)			X	X	
7. Writing-up and Documentation	Both				X	X

Tentative weeks: W1 = 3-9 Nov, W2 = 10-16 Nov, W3 = 17-23 Nov, W4 = 24-30 Nov, W5 = 1-7 Dec 2026. Development is planned to stop at the end of W4, apart from fixes needed for evaluation.

8.2 Project Outcomes and Deliverables

The expected deliverables are a working prototype, a labelled synthetic benchmark with its generator, a repeatable evaluation, a short demonstration and a final report. None of these has been achieved at this stage.

Table 8.3 maps the project outcomes to the strategic technology goals.

The template does not define these goals, so we set three for this project. Goal 1: Check invoices against orders, receipts and contracts with cited evidence. Goal 2: Keep AI use safe and accountable through human approval and controlled tool access. Goal 3: Evaluate the system in a repeatable way on synthetic data. The last column shows the objective each outcome is expected to meet.

Table 8.3 RELATIONSHIP TO STRATEGIC FRAMEWORK

PROJECT EXPECTED OUTCOMES	STRATEGIC TECHNOLOGY PROGRAM GOALS			PROJECT OBJECTIVE ACHIEVED
	GOAL 1	GOAL 2	GOAL 3	
Prototype workflow with citations and verification	X	X	-	1, 2, 3, 4 (expected)
Risk routing with human approval and vendor memory	X	X	-	5, 6 (expected)
Controlled tool access with simulated actions	-	X	-	7 (expected)
Labelled benchmark and evaluation report	-	-	X	8 (expected)
Final report and demonstration	-	-	X	1 to 8 (expected)

9. Utilization

The work will mainly serve an academic purpose, as a tested example of combining LLM agents, ordinary code and human review. Other students without real AP data could reuse the benchmark generator. The prototype would not be ready for operational use without testing on real documents and a proper security review.

10. Impact of the Work/Project

The project should produce a small, repeatable comparison of verification and retrieval choices, even if some results are negative. If later validated on real data, similar methods could reduce repetitive checking and help staff see the evidence behind each flag. This is only a possibility, not a measured benefit, and the project will not measure financial savings.

11. Sustainability

The project uses open-source tools, modular code and a generator that can recreate the benchmark. The main dependency is the LLM provider. Models and prices change, so provider calls will stay in one module. Results from a new model would have to be measured again. There is no funding or operating plan beyond the semester.

12. Governance and Ethical Aspect of the Project

Confidentiality and privacy. The project will use only synthetic documents and will not collect real company, vendor or customer data. Any future use with real data would need its own storage, access and retention controls.

Access control and traceability. Agents that read documents will have no action tools, and action tools will require an approval record. An audit log will record each finding, citation, score, decision and simulated action.

Accountability and human oversight. People, not the model, remain accountable for financial decisions. Reviewers will see the cited evidence with each finding, and low-risk cases will be randomly audited to limit over-reliance.

Limits of fraud detection. Automated flags are signals, not findings of wrongdoing. A false flag could delay payment to an honest vendor, and synthetic tests cannot show performance against real attackers. A person will make the final decision.

Prompt injection and responsible LLM use. Documents may contain malicious instructions, and no defence should be assumed to be complete. We will test this directly and report failures. The project will not claim regulatory compliance or certification.

13. Project Budget

Table 13.1 shows the proposed budget of the project and Table 13.2 provides the justifications of the budget. We have divided the project into two phases, first half and second half.

This is an unpaid student project, and we will not spend money on anything, so the total budget is 0.

Table 13.1 PROPOSED BUDGET

SUMMARY
PROPOSED BUDGET *Dollars ($)*

PROJECT TITLE		LedgerGuard: A Multi-Agent System for Invoice and Contract Reconciliation						
DURATION		**ABOUT 1.25 MONTHS (TENTATIVE)**						
ITEM	**CATEGORY**	**NO.**	**COMPENSATION**	**FIRST HALF**		**SECOND HALF**		**TOTAL**
				MONTHS	**BUDGET**	**MONTHS**	**BUDGET**	
A-Research Team	**CONSULTANTS**	0	None	-	0	-	0	0
	PRINCIPAL INVESTIGATOR	0	None	-	0	-	0	0
	CO-INVESTIGATOR-2	0	None	-	0	-	0	0
						TOTAL-A	0	0
B-Assistants	**PHD STUDENTS**	0	None	-	0	-	0	0
	MS STUDENTS	0	None	-	0	-	0	0
	UNDERGRADUATE STUDENTS	2	Unpaid	0.6	0	0.6	0	0
	TECHNICIANS	0	None	-	0	-	0	0
	SECRETARIAL-CLERICAL	0	None	-	0	-	0	0
	OTHER (programmers and developers)	0	None	-	0	-	0	0
						TOTAL-B	0	0
C-Computer Services & Consultants	**Computer Service** (Connection and telecommunication costs)			Internet connection and communication needs		0	0	0
	CONSULTANT	0	None	-	0	-	0	0
	Consultant Services	0	None	-	0	-	0	0
						TOTAL-C	0	0
D-EQUIP. & MATERIAL	**MAJOR EQUIPMENTS (<= 100,000)**			0		0		0
	EQUIPMENTS Major (> 100,000)			0		0		0
	EQUIPMENTS Minor (> 10,000)			0		0		0
	CONFERENCES			0		0		0
	PATENT REGISTRATION			0		0		0
	PUBLICATIONS			0		0		0
	WORKSHOP			0		0		0
	OTHER EXPENSES			0		0		0
ITEM TOTAL				0		0		0
GRAND TOTAL				0		0		0

A. SALARIES (Research Team)	0%	
B. SALARIES (Assistants)	0%	
C. Computer Services & Consultants	0%	
D. EQUIP. & MATERIAL	0%	
GRAND TOTAL	0%	

Table 13.2 BUDGET JUSTIFICATION

Resource	Category	Status and cost
Python, LangGraph, Pydantic, PyMuPDF, SQLite, MCP Python SDK, Streamlit, pytest	Free and open-source	No purchase needed
Pinecone	Free tier	No cost
Team members' own computers	Existing resource	Available
University facilities	None	None
Internet connection	Existing resource	Available, no cost to the project
LLM API usage	Free tier	Groq, no cost
Hosting and deployment	Optional free platform	Free platforms like HF if needed, no cost
Salaries, consultants, equipment, conferences, patents, publications, workshops	Not proposed	None

14. Declaration of AI usage.

Claude (Anthropic) was used to help prepare this proposal, including summarising sources and drafting and shortening the text, tables and workflow figure. The project idea, initial objectives and methodology came from the team, and the team reviewed the final document and remains responsible for its accuracy.

15. Conclusion

Invoice checking is a practical problem where both human and LLM mistakes can cost money. LedgerGuard proposes a limited prototype that keeps calculations in code, checks each finding against its evidence and leaves sensitive decisions to people. The evaluation will use synthetic data, so the expected result is a measured study of the design rather than a finished product.

REFERENCES

[1] T. Tater et al., "AI driven accounts payable transformation," in Proc. AAAI Conf. Artif. Intell., vol. 36, no. 11, 2022, pp. 12405–12413, doi: 10.1609/aaai.v36i11.21506.

[2] D. Thomas, D. Yadav, and B. T. Jin, "Beyond detection: A multi-agent framework for root cause analysis of financial discrepancies in distributed environments," presented at the KDD Workshop Agentic AI Enterprise, Toronto, ON, Canada, 2025. [Online]. Available: https://assets.amazon.science/69/cb/d93876ef4ce5b8c2a2a07087b261/beyond-detection-a-multi-agent-framework-for-root-cause-analysis-of-financial-discrepancies-in-distributed-environment.pdf